Computer Architecture Lecture 6

Sai Deepak Sana (cs23btech11055)
Arjun Konidala (cs23btech11026)

August 19th, 2024

Large Immediates

Loading a large (32-bit) immediate value in a register. (let's take an example **0x20112f4d**)

- Option 1: Directly store the value in memory and load using ld.
- Option 2: Using multiple arithmetic/shift operations

```
addi x6, x6, 0x201   // The last 12 bits will get filled with this 0x201
slli x6, x6, 12      // Then left shift these 12 bits
addi x6, x6, 0x12f   // And then again fill last 12 bits with next 3 hex digits
slli x6, x6, 8       // Left Shift these 12 digits by 8
addi x6, x6, 0x4d    // Then fill the last 8 digits by next 2 hex digits
```

- Option 3: Using **lui** (Load Upper Immediate) instruction

Upper 32 bits	Immediate 20 bits	Lower 12 bits

```
lui x19, 0x20112     // Here these 5 hex digits goes to the immediate 20 bits
addi x19, x19, 0xf4d // These 3 hex digits goes to the last 12 bits.
```

The immediate value used in lui instruction gets sign-extended to the upper 32-bits.

U-format **(Upper)**

This instruction format is used for instructions that use immediate values of size 20 bits.
1) **lui** (Load Upper immediate)
2) **auipc** (Add upper immediate to program counter): rd ← PC + offset created in similar way as lui.
Here PC is the program counter, pointing to the currently executing instruction.

The instruction encoding for U-format instructions is:

imm[31:12]	rd (5bits)	opcode (7bits)

Jump Addressing

The Jump and Link (jal) instruction has only one register operand. It takes both an immediate value and a register as operands. The instruction causes a jump to the address calculated by adding the immediate value to the program counter (PC) value of the jal instruction itself. The immediate value is always even to ensure proper jump so the least significant bit (bit 0) is ignored. As a result, 20 bits of the immediate value are used, specifically from [20:1], while the [0] bit is excluded since it is always 0.

Note: The immediate value must be defined with 21 bits to ensure proper execution.

UJ-format (Upper Jump)

The UJ-format is used for instructions that involve a jump to a target address specified by a 20-bit immediate value.

imm[20] sign bit	imm[10:1]	imm[11]	imm[19:12]	rd	opcode
1 bit	10 bits	1 bit	8 bits	5 bits	7 bits

- Unlike U-format, UJ -format instruction immediate value bits are jumbled in positions, because [10:1] bits are common with I-format which may favour regularity

The other jump instruction (jalr) follows the I-type encoding as it has one source register, one destination register, and one immediate offset.

Summary of instruction formats in RISC-V

Name (Field size)	Field						Comments
	7 bits	5 bits	5 bits	3 bits	5 bits	7 bits	
R-type	funct7	rs2	rs1	funct3	rd	opcode	Arithmetic instruction format
I-type	immediate[11:0]		rs1	funct3	rd	opcode	Loads & immediate arithmetic
S-type	immed[11:5]	rs2	rs1	funct3	immed[4:0]	opcode	Stores
SB-type	immed[12,10:5]	rs2	rs1	funct3	immed[4:1,11]	opcode	Conditional branch format
UJ-type	immediate[20,10:1,11,19:12]				rd	opcode	Unconditional jump format
U-type	immediate[31:12]				rd	opcode	Upper immediate format

RISC-V addressing summary

RISC-V has 4 main types of addressing :

1) Immediate addressing: The instruction itself contains an immediate operand.

2) Register addressing: The operand is located in registers. The instruction specifies the register number, allowing the operation to use the value stored in that register.

3) Base (or) displacement addressing: The operand is located in memory. The address of this operand is obtained by adding an immediate value to a base address stored in a register.

4) PC-relative addressing: The branch or jump target address is calculated by adding a constant value, specified in the instruction, to the current value of the program counter (PC). Such PC-relative addressing for branch and jump are helpful to migrate a piece of code from one address to another address without modifying the code.

Decoding Machine Code

Since assembly code to machine code mapping is 1-1 (unlike C code to assembly code), it is possible to convert a machine instruction to its equivalent assembly instruction. Example: Convert **0x00628533** to assembly code

- First Convert the hex code into binary i.e., (0000 0000 0110 0010 1000 0101 0011 0011)
- Then determine the Opcode, last 7 bits.(i.e., 0110011 **R-Type**) check it with the table
- Then match bits with corresponding fields.

funct7	rs2	rs1	funct3	rd	opcode
0000000	00110	00101	000	01010	0110011

- That is **add x10, x5, x6**

Summary of Instructions

- **Arithmetic** : add, sub, addi.
- **Logical** : and, or, xor, andi, ori, xori
- **Data Movement** : ld/w/h/b, sd/w/h/b
- **Shift** : sll, srl, sra, slli, srli, srai
- **Branch** : beq, bne, bge, blt, bgeu, bltu
- **Jump** : jal, jalr
- **Others** : lui, auipc

Memory Layout

The memory (RAM) allocated to a program is generally divided into four segments/sections:

- **Text:** This segment contains the executable instructions and any read-only constants. This is a read-only section in general.
- **Static Data:** This is where global and static variables in C are stored.
  - `x3` is typically used as the global pointer, initialized to the base address of the data segment, allowing for positive or negative offsets within this segment.
- **Dynamic Data:** This segment corresponds to the heap and stores the dynamically allocated memory.
- **Stack:** This segment is used for allocation of automatic or local variables (scalar or array or structures). The stack grows and shrinks as we enter or come out from a function.
  - In general, The heap begins at the lower addresses of this segment and grows upwards.
  - The stack starts from the higher addresses and grows downwards.
  - This allows flexibility in terms of whether a program uses more stack or more heap.
- **Reserved Data:** Certain memory regions are reserved for other system operations.

Storage Classes in C

1) auto int a, b, c; :
These variables are stored in the stack and exist until the end of the block or function. This is the default storage class, if no other storage class is specified.

2) register int a, b, c; :
In the program, we give a hint/request to the compiler to store `a, b, c` in the CPU registers. However, it is ultimately the compiler's decision to generate assembly code that may store the variables either in registers or in the stack. These variables also exist until the end of the block or function.

3) static int a, b, c; :
These variables are stored in the data segment but can only be accessed within the specific scope of a block or function. They exist until the end of the program.

4) extern int a, b, c; :
These variables are also stored in the data segment and can be accessed by other instructions at any scope. Their lifetime is the end of the program.

Procedure/Funtion calls

Transfer of Control

- **Passing the arguments:** The process of providing input data or parameters to the called function.
- **Jumping to the function:** The control is transferred to the function's entry point to begin execution.
- **Performing the computation:** The function does the intended computation.
- **Passing back the results (if any):** The function returns the output or result back to the calling point.
- **Returning to the original point:** Control returns to the next instruction after the function call in the main program.

Variables inside a Function

- **Automatic:** Local variables created within the function, usually stored in the stack and deallocated once the function exits.
- **Static: global pointer (gp/x3):** Persistent variables that maintain their value across multiple function calls, often managed through a global pointer in low-level implementations.

Types

- **Leaf procedure:** A function that does not call any other functions.
- **Nested procedure:** A function that calls another function within its scope.
- **Recursive procedure:** A function that calls itself to solve a problem by breaking it into smaller sub problems.

Procedure Calling

Procedure (or) functions are used in programs to structure programs in easier way. In execution of procedure the program must follow these 6 steps

1. Place parameters in a pre-defined location.
2. Transfer control to the procedure.
3. Acquire storage for the procedure.
4. Perform the procedure's operations.
5. Place the result in a pre-defined location for the caller.
6. Return to the place of call.

Procedure Call Instructions

Procedure Call: jump and link

```
jal x1, ProcedureLabel // jumps to target address and stores address of next instruction in x1
```

Procedure return: jump and link register

```
jalr x0, 0(x1) // jumps to location (0 + x1). rd is x0 and hence write to it is ignored.
```

Using more registers in procedure calling

What happens if a function requires a large number of registers and also needs to restore the previous register values necessary for the program's execution?

- In this scenario, registers need to be spilled to memory. The ideal data structure for spilling registers is the stack
- A stack requires a stack pointer to keep track of the most recently allocated address, indicating where the next procedure should store spilled registers or where to find saved registers.
- In RISC-V, the stack pointer (sp) is generally stored in the register x2.

Example: Now lets see how a basic leaf procedure gets in stack.

```
//int64_t refers to 64-bit int
int64_t leaf_example (int64_t g, int64_t h, int64_t i, int64_t j)
             {
               int64_t f;
               f = (g + h) - (i + j);
               return f;
             }
```

In this function, there is a need to use some registers that were previously used. To restore the original values of these registers, the old values must be saved onto the stack before they are overwritten.
Since the function uses x5, x6, and x20 for storing some values and overwrites their original content, it saves them first and then restores at the end. In order to save them, it creates a space in the stack by decrementing the stack pointer by 24 bytes (3 x 8 Byte storage).

```
leaf_example:
    addi sp, sp, -24   //shifting the stack pointer
    sd x5, 16(sp)      //storing old values of x5
    sd x6, 8(sp)       //storing old values of x6
    sd x20, 0(sp)      //storing old values of x20
```

```
add x5, x10, x11   //using x5 temporarily
add x6, x12, x13   //using x6 temporarily
sub x20, x5, x6    //loading difference to x20 which is also temporary
addi x10, x20, 0   //in general return values and arguments in RISC-V use x10-17 registers
ld x20, 0(sp)      //restoring values as they are before the call
ld x6, 8(sp)       //restoring values as they are before the call
ld x5, 16(sp)      //restoring values as they are before the call
addi sp, sp, 24    // updating the sp
jalr x0, 0(x1)     // return to instruction next to caller
```